

Executing the Enterprise Continuum: Diagnostic Phase Space and STOP Regions

1 Scope and Reproducibility

1.1 Document class and intent

This document is an *executable demonstration* of diagnostic *phase space* construction and *STOP regions* for the Enterprise Continuum K_{EA} . It is strictly *non-prescriptive*: it does not recommend interventions, policies, or managerial actions.

The present PDF-1 is a *container skeleton*. It provides a stable LaTeX structure for later computational demos (in subsequent runs), while preserving the immutability of: (i) KEA (PDF-0) and (ii) ETS.K_EA.v1.1. No new theoretical primitives are introduced here, and no semantic drift is permitted.

1.2 Immutability and scope guardrails

- **Immutable inputs:** KEA (PDF-0) and ETS.K_EA.v1.1 are treated as authoritative.
- **No new primitives:** only re-use of existing definitions and terms.
- **No engine in RUN-EDHQ-01:** no computation, no implementation details, no algorithms beyond placeholders.
- **No pipeline edits:** the shared preamble and build tooling remain untouched.

1.3 Reproducible build path

The build must succeed when executed from the repository root using PowerShell and `latexmk`. The canonical local build commands are:

```
Set-Location C:\Users\Megaport\work\ea-oc-extension
Push-Location .\paper\pdf1_executing_kea
latexmk -pdf -interaction=nonstopmode -file-line-error -halt-on-error '
.\main.tex
Pop-Location
```

Expected output:

- PDF artifact: `paper/pdf1_executing_kea/main.pdf`
- Exit status: 0 (success)

1.4 What this PDF-1 will contain (structural)

Subsequent sections provide only the structural slots for: (i) phase space construction, (ii) synthetic regimes, (iii) executing diagnostics, (iv) STOP regions, (v) invariance stress tests,

and (vi) explicit non-claims and limits. Any future computational content must remain diagnostic-only and must not modify the KEA/ETS core.

2 Phase Space Construction

2.1 Objective of the construction

This section defines the *structural container* for constructing a minimal, executable, geometry-only phase space over the Enterprise Continuum K_{EA} .

The goal of RUN-EDHQ-02 is strictly limited to producing a reproducible visualization of the admissible state region $\Omega(K_{EA})$ as specified by ETS.K_EA.v1.1, without introducing any new variables, metrics, primitives, diagnostic claims, or phase transition logic.

All coordinates, axis names, and numerical bounds referenced here are inherited *only* from KEA (PDF-0) and ETS.K_EA.v1.1. No extensions are permitted in this run.

2.2 State representation (container)

An enterprise state is represented as a point (or region) in the admissible phase space defined by:

- the existing state descriptors of K_{EA} ,
- the admissible configuration set $\Omega(K_{EA})$ as specified by the ETS,
- explicit boundary conditions given by the ETS domain constraints.

This subsection intentionally omits any semantic interpretation, diagnostics, or claims about realizability beyond the explicit ETS constraints. It exists solely to anchor executable demonstrations to a stable formal slot.

2.3 Admissible regions and boundaries

The phase space is partitioned into admissible and non-admissible regions according to constraints already defined in the immutable sources (KEA, ETS). Boundary surfaces are treated as formal *domain boundaries* of $\Omega(K_{EA})$, with no additional assumptions about smoothness, convexity, continuity, or optimality beyond what is explicitly stated in the underlying specification.

2.4 Observability and projection (reserved slot)

Observed enterprise signals (as defined in the immutable sources) may be mapped into the phase space by projection rules defined elsewhere. This document does not define any new projection operators; it only reserves the structural location where such projections may be referenced.

2.5 Executable minimal phase space demo (RUN-EDHQ-02)

A minimal, reproducible, geometry-only demonstration is implemented as a Python script:

- `src/pdf1/phase_space_demo.py`

which loads `ETS.K_EA.v1.1`, extracts axis names and their numerical bounds as given by the ETS, selects a deterministic 2D projection (by a fixed rule implemented in the script), and generates a single phase-space figure representing the admissible region $\Omega(K_{EA})$ under those bounds.

All run parameters (seed, grid size, axis ranges, script hash, and produced artifacts) are captured in:

- `results/pdf1/phase_space_demo/run_manifest.json`

2.6 Generated figure (geometry only)

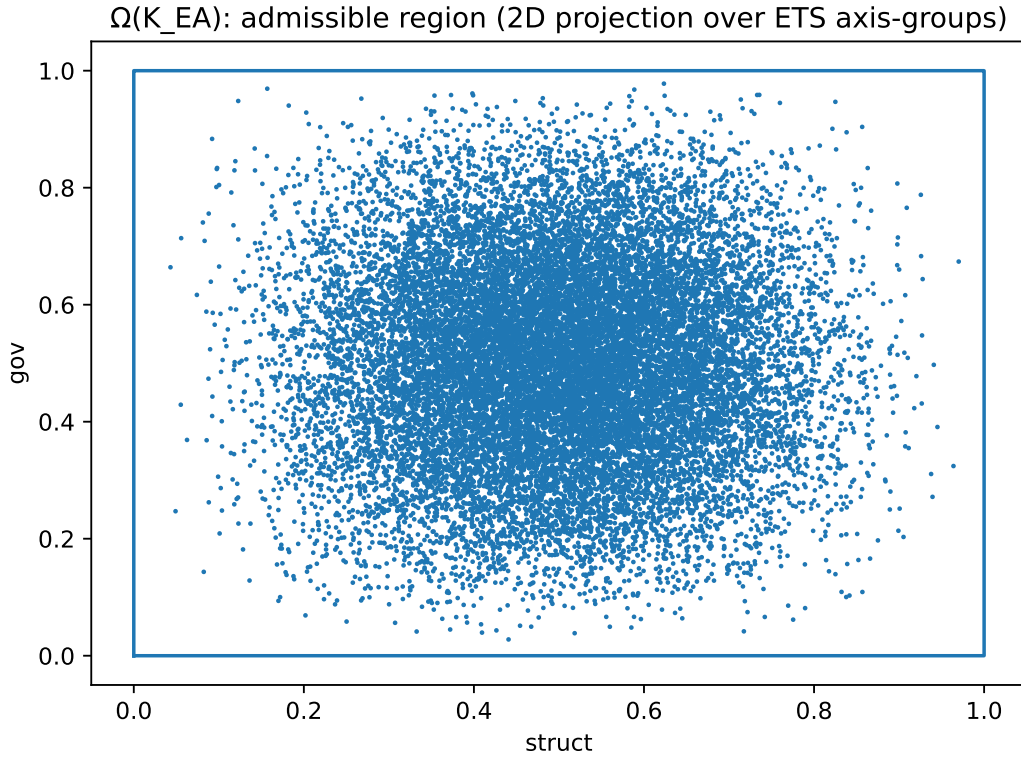


Figure 1: Admissible region $\Omega(K_{EA})$ (2D projection) generated from `ETS.K_EA.v1.1`.

2.7 Non-goals

- No STOP logic.
- No diagnostics, classification, or phase logic.
- No construction of new dimensions or axes.
- No optimization criteria, targets, or control objectives.
- No claims about completeness or sufficiency of observation.

3 Synthetic Regimes

3.1 Purpose of synthetic regimes

Synthetic regimes are used here as *diagnostic abstractions* for grouping regions of the enterprise phase space that exhibit structurally similar behavior under observation.

They are not empirical clusters, forecasts, or managerial categories. Their sole purpose is to provide a stable descriptive layer for reasoning about diagnostic outcomes already implied by the KEA framework.

3.2 Regime definition (container)

A synthetic regime denotes a subset of the admissible phase space characterized by:

- shared structural constraints,
- comparable boundary proximity with respect to STOP regions,
- invariant qualitative behavior under admissible diagnostic projections.

All regime boundaries and descriptors are inherited from existing KEA/ETS definitions. No new regime taxonomy is introduced in this document.

3.3 Relation to phase space

Each synthetic regime occupies a well-defined region (possibly non-convex or disconnected) within the constructed phase space. The mapping between regimes and phase space regions is many-to-many and purely descriptive.

This section intentionally avoids any ranking, scoring, or prioritization of regimes.

3.4 Stability and transitions (non-prescriptive)

Observed transitions between regimes are treated as *diagnostic signals*. They do not imply controllability, desirability, or reversibility.

No transition rules, probabilities, or causal mechanisms are specified here.

3.5 Explicit exclusions

- No lifecycle or maturity models.
- No success/failure labeling.
- No optimization paths between regimes.
- No implied intervention recommendations.

4 Executing Diagnostics

4.1 Role of execution in a diagnostic context

Execution in this document refers exclusively to the *procedural application* of already defined diagnostic rules to observed enterprise states. It does not imply intervention, control, optimization, or decision enforcement.

The term “executing” is used in a strictly technical sense: applying an immutable specification to concrete observations in a reproducible manner.

4.2 Diagnostic inputs (container)

Diagnostic execution assumes the availability of:

- observed enterprise signals admissible under the KEA scope,
- a fixed mapping into the diagnostic phase space,
- invariant boundary and STOP conditions as defined in the ETS.

No assumptions are made about data completeness, quality, or timeliness.

4.3 Execution flow (structural slot)

This subsection reserves the structural slot for a future step-by-step diagnostic flow, such as:

1. projection of observations into phase space,
2. identification of the occupied synthetic regime(s),
3. evaluation of proximity to STOP regions,
4. generation of diagnostic outcomes.

In RUN-EDHQ-01, this flow is intentionally non-operational and non-algorithmic.

4.4 Determinism and reproducibility

Given identical inputs and an unchanged specification, diagnostic execution must be deterministic and reproducible. Any future executable realization must preserve this property.

4.5 Non-claims

- No guarantee of actionability.
- No claim of predictive power.
- No promise of real-time execution.
- No implication of managerial decision support.

5 STOP Regions

5.1 Conceptual role of STOP regions

STOP regions represent formally defined boundaries in the enterprise phase space beyond which diagnostic execution must halt. They are first-class diagnostic outcomes, not errors and not recommendations.

STOP regions encode structural impossibility, loss of admissibility, or violation of invariant constraints already specified in the KEA and ETS layers.

5.2 STOP as a diagnostic outcome

Reaching a STOP region signifies that further diagnostic refinement is either undefined or non-falsifiable under the given specification. This outcome is explicit, terminal, and non-negotiable.

STOP does not imply failure, blame, urgency, or the necessity of action.

5.3 Executable STOP mask (2D demo)

This section includes a minimal executable demonstration of STOP regions as a binary mask (admissible vs. STOP) over a deterministic 2D projection of the ETS axis-groups.

The STOP predicate is taken verbatim from the immutable ETS. All execution parameters, hashes, and the explicit tooling-only instantiation used for this demo are recorded in the run manifest.

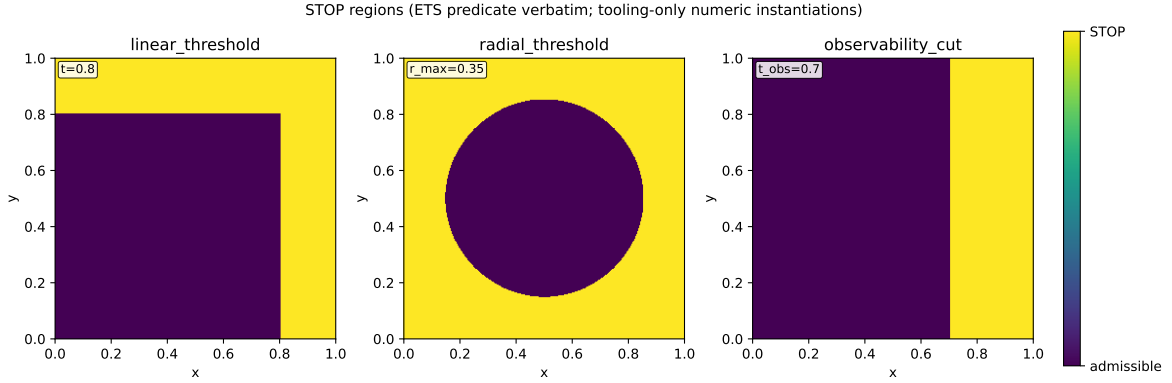


Figure 2: Executable STOP regions mask over a deterministic 2D projection under multiple tooling-only instantiations (ETS predicate verbatim; instantiation recorded in manifest).

5.4 Boundary characterization (container)

STOP regions are characterized by:

- violation of admissibility constraints,
- collapse of observability assumptions,
- incompatibility with invariant stress conditions.

No new boundary criteria are introduced here. All references are strictly inherited from immutable sources.

5.5 Detection and signaling

This section reserves the structural location for detecting proximity to, or entry into, STOP regions during diagnostic execution. Signaling of STOP is treated as a formal result, not a warning or alert.

5.6 Explicit exclusions

- No recovery strategies.

- No mitigation or workaround logic.
- No escalation or governance prescriptions.
- No optimization away from STOP.

6 Invariance Stress Tests

6.1 Purpose of invariance testing

Invariance stress tests are used to verify that diagnostic outcomes remain stable under admissible transformations of observation, representation, or projection. They do not test performance, efficiency, or effectiveness.

The objective is to detect violations of invariants already defined in the KEA and ETS layers.

6.2 Classes of admissible perturbations

This section reserves space for discussing admissible perturbations, such as:

- reparameterization of observation without semantic change,
- partial loss or aggregation of signals within admissible bounds,
- alternative but equivalent projection paths.

No new perturbation classes are defined here.

6.3 Stress test outcomes

Possible outcomes of invariance stress tests include:

- confirmed invariance (diagnostic stability),
- detected sensitivity (boundary proximity),
- violation leading to STOP region identification.

All outcomes are descriptive and diagnostic-only.

6.4 Relation to falsifiability

Failure of invariance under admissible conditions is treated as a falsification signal for the diagnostic setup under the current observation context. This does not falsify the KEA itself, but constrains its applicability.

6.5 Non-goals

- No robustness optimization.
- No tolerance tuning.
- No resilience scoring.
- No operational recommendations.

7 Non-Claims and Limits

7.1 Explicit non-claims

This document makes the following explicit non-claims:

- No prescriptive guidance for enterprise change or intervention.
- No optimization objectives or target states.
- No performance benchmarks, KPIs, or maturity models.
- No predictive guarantees or probabilistic forecasts.
- No managerial, governance, or policy recommendations.

7.2 Limits of applicability

The diagnostic constructions presented here are applicable only within the formal scope defined by KEA (PDF-0) and ETS.K_EA.v1.1. Outside this scope, diagnostic results may be undefined, non-falsifiable, or collapse into STOP regions.

This section does not attempt to extend applicability beyond formally specified boundaries.

7.3 Epistemic status

All results obtained via this PDF-1 are diagnostic descriptions under explicit assumptions. They do not constitute explanations, causes, or actionable knowledge.

Uncertainty, partial observability, and model mismatch are treated as first-class limiting factors, not as defects to be corrected.

7.4 Relation to future runs

Any future extension (e.g., computational demos, executable traces) must:

- preserve all non-claims stated here,
- remain compatible with invariant stress tests,
- avoid introducing implicit prescriptions.

7.5 Closure

This section serves as a hard boundary for interpretation. Crossing this boundary converts the document into a different class of artifact, which is explicitly out of scope for PDF-1.

A Reproducibility Manifest

A.1 Purpose

This appendix specifies the minimal reproducibility contract for PDF-1. It documents the conditions under which the document can be built, inspected, and verified without modifying the theoretical core or the build pipeline.

The manifest is descriptive and declarative; it does not introduce executable logic.

A.2 Artifact identity

- **Artifact class:** Executable Demonstration / Diagnostic / Non-prescriptive
- **Artifact ID:** PDF-1 — Executing the Enterprise Continuum
- **Immutable sources:** KEA (PDF-0), ETS.K_EA.v1.1

A.3 Build environment assumptions

The following assumptions define the supported local build environment:

- Operating system: Windows
- Shell: PowerShell
- Build tool: `latexmk`
- TeX distribution: any standard distribution compatible with the shared preamble

No guarantees are made for alternative shells, operating systems, or modified pipelines.

A.4 Canonical build command

The canonical reproducible build is defined by the following command sequence, executed from the repository root:

```
Set-Location C:\Users\Megaport\work\ea-oc-extension
Push-Location .\paper\pdf1_executing_kea
latexmk -pdf -interaction=nonstopmode -file-line-error -halt-on-error '
.\main.tex
Pop-Location
```

A successful build must terminate with exit status 0 and produce `paper/pdf1_executing_kea/main.pdf`.

A.5 Scope integrity guarantees

This manifest guarantees that:

- no new theoretical primitives are introduced,
- no existing definitions are modified,
- no pipeline code or shared preamble is altered,
- all content remains diagnostic-only.

Any deviation from these conditions invalidates the reproducibility claim.

A.6 Non-guarantees

The following are explicitly not guaranteed:

- numerical stability or performance characteristics,
- completeness of observation data,
- applicability outside the KEA/ETS formal scope,
- suitability for decision-making or intervention.

A.7 Verification status

This appendix defines the criteria for reproducibility but does not assert that a particular build has been executed. Verification must be established by an explicit build log and commit hash.